Name: Fazal Elahi

Reg no. L1F22BSDS0033

Name: M. Khallad Ahmad

Reg no. L1F22BSDS0020

Name: Azhar Sajjad

Reg no. L1F22BSDS0021

Instructor: M. Rizwan

Project: TRAVEL GUIDE RAG

1. Importing Necessary Libraries

It starts by importing many of the core Python libraries like os, json, re, and time. These are the most basic libraries required for file operations, working with JSON data, regular expression usage, and timing execution during processing phases. Moreover, higher-level libraries like nltk, transformers, and BeautifulSoup are utilized for natural language processing and HTML parsing, which are needed for working with and converting raw text data.

2. Web Scraping and Content Extraction

It does web scraping to gather travel-related articles from web sources with a structure like nomadicMatt, guidetoPakistan, wikivoyage, each of them corresponding to a particular destination or topic. On every file, it opens the content, parses and extracts clean textual data using BeautifulSoup, and removes unnecessary characters or formatting using regular expressions. Scraping pipeline guarantees that informative content like article bodies, titles, and descriptions are gathered but without navigation links, ads, and boilerplate information. This is important to produce a high-quality dataset specific for question answering as well as information retrieval.

3. Initialization of Text Preprocessing Tools

The script initializes different parts of the NLTK library such as the list of stopwords and lemmatizer. Stopwords are frequent words (such as "the", "and", "is") that are normally discarded to minimize noise. Lemmatization is used to turn words into their base or root form (i.e., "running" to "run"), enhancing uniformity and efficiency during search and retrieval.

4. Loading and Parsing Input Data

Multiple JSON or JSONL files with travel-related content are loaded via Python's json module. The documents are organized with fields like title and text. The loading functions convert them to dictionaries or lists, ready for preprocessing. The data structure is maintained so that metadata (e.g., titles) is available for use in retrieval or answer generation.

5. Complete Preprocessing Workflow

Two different preprocessing functions are used in separate phases of the pipeline—one for documents and another for queries. For document preprocessing, the raw text is first converted to lowercase to ensure uniformity and reduce vocabulary variations. Special characters and punctuation are removed using regular expressions, followed by tokenization with NLTK. Stopwords are then eliminated to discard common but uninformative words. Lemmatization reduces words to their base form, enhancing semantic understanding. Additional cleaning includes removing patterns, artifacts, and extra whitespace. The query preprocessing pipeline is similar but tailored for short, user-generated input. It ensures queries are clean, concise, and semantically enriched before embedding and matching. Both processes include validation steps—documents are checked for adequate length, while queries are filtered for noise. This consistent approach ensures both the indexed content and input queries are optimized for retrieval.

6. Text Chunking for Long Documents

Because most documents are longer than the largest input size of transformer models, the notebook provides a function split_into_chunks to split long texts into overlapping smaller segments. The segments maintain semantic continuity across boundaries, which is relevant for both summarization and retrieval tasks.

7. Document Embedding and Vectorization

With the SentenceTransformer model (usually a pretrained transformer model such as **multi-qa-mpnet-base-dot-v1**), every text chunk is represented as a dense vector representation. The vectors encode the meaning of the text, enabling similarity comparisons across queries and documents.

8. Creating and Utilizing the FAISS Index

After embedding all the documents, the FAISS library is utilized to construct an efficient index. This supports rapid approximate nearest neighbor search. Embedding normalization prevents cosine similarity from being invalid. The system returns the top-k most similar documents by vector proximity to the query.

9. Query Expansion through Relevance Modeling

Following the initial retrieval, a query expansion module is used. It uses the highest-retrieved documents and detects high-probability or contextually important terms to use in expanding the original query. This is done to assist the system in retrieving relevant documents that are not exactly matching the vocabulary of the original query.

10. Summarization of Retrieved Content

A summarization function based on **facebook/bart-large-cnn** takes the highest ranked retrieved document pieces and condenses their content into a short paragraph. The summary extracts the main points of the retrieved material and assists in enhancing the subsequent retrieval process.

11. Re-retrieval Using Summary

The system reuses the generated summary as a refined query to perform a second, more focused retrieval. The rationale behind this approach is that the initial query might be too generic, ambiguous, or limited in scope. By summarizing the top retrieved chunks into a coherent paragraph, the model effectively transforms multiple relevant ideas into a single, context-rich input. This new query—being more representative of the intended information need—is then passed back to the retrieval engine. During this second pass, a smaller and more relevant subset of documents is retrieved, which not only enhances the precision of the answer generation process but also reduces noise from unrelated content. This iterative improvement of the query reflects the principles of relevance feedback and helps the system converge toward higher-quality responses.

12. Saving Processed Chunks

The system can save the processed and chunked data in .jsonl format. Every entry includes metadata like title and the cleaned text chunk. This helps in caching and reusing intermediate results for future runs.

13. Execution Pipeline and Output

In the last part within the if __name__ == "__main__" block, the entire RAG pipeline is run. It entails loading documents, preprocessing, chunking, embedding, indexing, retrieval, summarization, and re-retrieval. It manages input queries, prints the final answers, and logs performance metrics like time taken for each major step.

14. Summary of System Architecture

The entire architecture is a modular and extensible Retrieval-Augmented Generation (RAG) pipeline. It incorporates best practices in document cleaning, embedding-based retrieval, language modeling, and summarization. The individual components all work together to turn unstructured raw data into precise, context-sensitive answers to user questions. The architecture supports scalability and reuse and can be applied to knowledge-intensive NLP tasks.